



Security Review For TokenLogic



Collaborative Audit Prepared For:
Lead Security Expert(s):

TokenLogic

0x52

pkqs90

Date Audited:

February 24 - February 26, 2026

Introduction

GHO is a decentralised, overcollateralised stablecoin that is fully backed, transparent, and native to the Aave Protocol. sGHO is a GHO vault that offers depositors the Aave Savings Rate without exposing them to protocol risk. The sGHO contract is a typical ERC4626 implementation with a few modifications. Users deposit assets and get shares based on the current yield index. Over time, the yield index will grow based on the target rate set by the yield manager which allows users to receive more assets than deposited.

Scope

Repository: `aave-dao/gho-origin`

Audited Commit: `f46868277c5e8b715cb33dcd6564e98cb73d064f`

Final Commit: `646ab32b290b0dd34934c867a69b26579a9b3ef4`

Files:

- `src/contracts/misc/sGhoSteward.sol`
- `src/contracts/sgho/interfaces/ISgho.sol`
- `src/contracts/sgho/sGho.sol`
- `tests/misc/sGhoSteward.t.sol`
- `tests/unit/TestSGhoBase.t.sol`
- `tests/unit/TestSGhoConfig.t.sol`
- `tests/unit/TestSGhoERC20.t.sol`
- `tests/unit/TestSGhoERC4626.t.sol`
- `tests/unit/TestSGhoGHOTokenAvailability.t.sol`
- `tests/unit/TestSGhoInitialization.t.sol`
- `tests/unit/TestSGhoPause.t.sol`
- `tests/unit/TestSGhoPermit.t.sol`
- `tests/unit/TestSGhoPrecision.t.sol`
- `tests/unit/TestSGhoRescue.t.sol`
- `tests/unit/TestSGhoYield.t.sol`

Final Commit Hash

`646ab32b290b0dd34934c867a69b26579a9b3ef4`

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	2

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue I-1: Configured 50% rate can realize about 64.87% annual yield under frequent updates [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-tokenlogic-feb-24th-2026/issues/16>

Vulnerability Detail

sGho exposes `setTargetRate()` in bps, converts it to `ratePerSecond`, then updates `yieldIndex` in `_getCurrentYieldIndex()` using a linear step over elapsed time.

Because `_update()` is called on every share movement (deposit/withdraw/mint/redeem/transfer), the linear step compounds across many updates.

```
function setTargetRate(uint16 newRate) public onlyRole(YIELD_MANAGER_ROLE) {
    uint256 annualRateRay = (uint256(newRate) * RAY) / 10000;
    $.ratePerSecond = (annualRateRay / 365 days).toUint96();
}

function _getCurrentYieldIndex() internal view returns (uint176) {
    uint256 accumulatedRate = $.ratePerSecond * timeSinceLastUpdate;
    uint256 growthFactor = RAY + accumulatedRate;
    return (($.yieldIndex * growthFactor) / RAY).toUint176();
}
```

At `newRate = 5000`, with updates every 12 seconds for one year:

- `ratePerSecond = 15854895991882293252`
- `step factor = 1.000000190258751902587519024`
- `yearly factor = step2,628,000 = 1.648721192279...`
- `effective APY = 64.872119...%` (not 50%)

Root cause: the configured annual rate is applied as a repeatedly compounded per-interval growth process, so effective annual yield depends on update frequency.

Impact

This can cause treasury/yield-budget mismatch and larger GHO shortfall pressure.

Issue I-2: Role documentation and code mismatch [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-tokenlogic-feb-24th-2026/issues/17>

Vulnerability Detail

sGho documentation describes PAUSE_GUARDIAN_ROLE as controlling deposits/withdrawals and YIELD_MANAGER_ROLE as controlling target rate updates.

Role Management

- PAUSE_GUARDIAN_ROLE : This role has permissions to pause/unpause sGho deposits and withdrawals
- TOKEN_RESCUER_ROLE : This role has permissions to rescue tokens held on the contract
- YIELD_MANAGER_ROLE : This role has permissions to update the yield target rate

However, implementation scope is broader:

1. Pause is enforced in `_update()`, which is the ERC20 movement hook. So pause also blocks `share transfer()`, `transferFrom()`, `mint/burn` paths.

```
function _update(address from, address to, uint256 value) internal override
↳ whenNotPaused {
    _updateYieldIndex();
    super._update(from, to, value);
}
```

2. YIELD_MANAGER_ROLE can update `supplyCap` directly on sGho, not only `targetRate`.

```
function setSupplyCap(uint160 newSupplyCap) public onlyRole(YIELD_MANAGER_ROLE) {
    _getSGhoStorage().supplyCap = newSupplyCap;
}
```

Impact

Emergency operations may incorrectly assume pausing only affects vault entry/exit, while transfers are also frozen.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.